

Generative Artificial Intelligence

Skill Development Tasks (1 to 15)

Task 1: Vectorized Scaled Dot-Product Attention from Scratch

- **Objective:** Master the mathematical foundation of modern Transformer models by deriving and implementing the core self-attention mechanism using optimized linear algebra operations.
- **Required Tech Stack:** Pure NumPy, Python 3.10+, Jupyter Notebook.
- **Task Description:** Students must implement the scaled dot-product attention formula
$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$
 from scratch. The code must handle batch-wise processing of multi-head inputs, casual sequence masking for decoder simulation, and scaling factors using raw 4D matrix manipulations without relying on high-level deep learning wrappers.

Task 2: Custom Byte-Pair Encoding (BPE) Tokenizer & Autoregressive Causal LM

- **Objective:** Understand the mechanics of subword tokenization and autoregressive sequence generation by building a production-grade language model training loop from the ground up.
- **Required Tech Stack:** PyTorch (Tensor operations only), raw Python, NumPy.
- **Task Description:** Students will write a custom Byte-Pair Encoding (BPE) vocabulary construction algorithm to merge high-frequency token pairs on custom text corpora. They must then construct a causal language modeling training loop, implementing custom causal masking matrices to prevent look-ahead leaks during generation.

Task 3: Local 7B Parameter LLM Quantization & Logit Extraction Pipeline

- **Objective:** Leverage cutting-edge model compression techniques to run large language models on edge devices, while building real-time interpretability hooks to inspect layer-wise logits and attention weights.
- **Required Tech Stack:** Ollama, Llama 3-8B-Instruct (GGUF), PyTorch, Hugging Face Transformers.
- **Task Description:** Deploy a local Llama-3-8B model using 4-bit integer quantization (Q4_K_M). Students will write an analytical extraction pipeline in PyTorch to tap into the local model's hidden layers, extracting token logit distributions and calculating dynamic entropy bounds across generation steps.

Task 4: In-Memory Hierarchical Navigable Small World (HNSW) Vector Indexing from Scratch

- **Objective:** Develop a high-speed vector retrieval engine to support semantic search and real-time knowledge retrieval without utilizing pre-built database drivers.
- **Required Tech Stack:** Python, NetworkX, SciPy, NumPy.
- **Task Description:** Students must mathematically construct a Hierarchical Navigable Small World (HNSW) graph for multi-layer vector indexing from scratch. The algorithm must construct a multi-tier proximity graph, execute probability-driven layer skipping during index traversal, and output cosine similarity rankings under $O(\log N)$ search complexity.

Task 5: Custom Transformer Backpropagation & Multi-Head Gradient Tracking Engine

- **Objective:** Analyze gradient flow and solve vanishing/exploding gradient problems within attention layers by executing manual backward-pass calculus.
- **Required Tech Stack:** PyTorch (with Autograd explicitly disabled via `requires_grad=False`), NumPy.
- **Task Description:** Students will implement a custom backward pass for a single-head self-attention layer. They must manually compute the partial derivatives of the loss function with respect to the Query, Key, and Value projection weight matrices ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$), tracking and graphing gradient magnitudes across simulated sequence lengths.

Task 6: High-Throughput Asynchronous RAG Pipeline with Cross-Encoder Reranking

- **Objective:** Construct an enterprise-ready Retrieval-Augmented Generation (RAG) system with complex dual-stage retrieval and real-time asynchronous routing.
- **Required Tech Stack:** FastAPI, LangChain, FAISS, PyTorch, SentenceTransformers, Asyncio.
- **Task Description:** Build a RAG service that handles asynchronous client queries. The system must retrieve candidate chunks using a bi-encoder embedding model, run an in-memory cross-encoder reranking model to filter irrelevant contexts, and orchestrate concurrent LLM synthesis under strict latency thresholds.

Task 7: Low-Rank Adaptation (LoRA) Matrix Projection Fine-Tuning of a 3B Model

- **Objective:** Perform parameter-efficient domain adaptation on medium-scale models, mathematically constraining weights to low-rank subspaces to minimize update

footprints.

- **Required Tech Stack:** PyTorch, Hugging Face PEFT, Transformers, CUDA Toolkit.
- **Task Description:** Students will design a custom LoRA modification block that inserts rank-decomposed parameter matrices ($\mathbf{A}$ and $\mathbf{B}$, where the rank $r \leq 8$) parallel to the attention projection weights. They must implement the scaling coefficient $\frac{\alpha}{r}$ and write a script to freeze base model parameters while optimizing only the low-rank adapters on domain-specific datasets.

Task 8: Multi-Agent Collaborative Swarm with Shared Transactional Blackboard Architecture

- **Objective:** Architect complex, multi-agent cooperative workflows to solve multi-step analytical problems, utilizing decentralized consensus and shared memory stores.
- **Required Tech Stack:** AutoGen, Docker-Compose, PostgreSQL, Redis, OpenAI API/Ollama.
- **Task Description:** Deploy a containerized multi-agent system where independent specialist agents (e.g., Code Generator, System Auditor, QA Analyst) negotiate via a shared Redis-backed Blackboard memory model. Agents must programmatically lock state keys, resolve execution deadlocks, and commit final outputs to a transactional database.

Task 9: Real-Time Vector Stream Ingestion & Spark-Driven Dynamic Reindexing

- **Objective:** Handle high-velocity, unbounded data streams by vectorizing continuous payloads and dynamically updating high-dimensional indices.
- **Required Tech Stack:** Apache Kafka, PySpark, Qdrant Vector DB, sentence-transformers.
- **Task Description:** Set up a local Kafka cluster streaming real-time transactional or news logs. Students must configure PySpark to process streaming mini-batches, apply a vectorization model across the streaming partitions, and execute upsert queries to a vector database with zero downstream query downtime.

Task 10: Denoising Diffusion Probabilistic Model (DDPM) Forward & Reverse Latent Optimization

- **Objective:** Build deep generative models for image synthesis from first principles, studying the transition from raw Gaussian noise to structured pixels.
- **Required Tech Stack:** PyTorch, Diffusers, NumPy, Matplotlib.
- **Task Description:** Students will mathematically formulate the forward noise schedule $q(x_t|x_{t-1})$ using a linear variance scheduler. They must then write the reverse

generative denoising loop $p_{\theta}(x_{t-1}|x_t)$ from scratch, generating 64×64 resolution image arrays iteratively over 1000 timesteps.

Task 11: Prompt Injection Guardrails & Adversarial Attack Simulation Engine

- **Objective:** Engineer resilient security architectures for LLM deployments by designing real-time semantic sanitization and adversarial detection middleware.
- **Required Tech Stack:** NeMo Guardrails, PyTorch, LangChain, Transformers.
- **Task Description:** Build an automated evaluation suite that simulates injection attacks (such as system prompt extraction and indirect payload execution). Students must write custom semantic classifiers that run alongside the prompt execution loop to detect, flag, and intercept adversarial prompts before they reach the foundation model.

Task 12: Knowledge Distillation of Dual-Encoder Vision-Language Models

- **Objective:** Compress massive multi-modal representation systems into highly efficient edge-deployable feature extractors using soft-label Kullback-Leibler (KL) divergence.
- **Required Tech Stack:** PyTorch, Hugging Face Transformers, TorchVision, SciPy.
- **Task Description:** Students will load a dual-encoder CLIP model as a static "Teacher." They must construct an uninitialized, lightweight "Student" model and train it using a custom composite loss function consisting of a weighted average of traditional cosine distance loss and KL-divergence on logit layers over high-resolution image-text datasets.

Task 13: Direct Preference Optimization (DPO) and Pairwise Reward Alignment

- **Objective:** Align model behavior with targeted human values and system safety guidelines using contrastive pairwise preference tuning without RL complexity.
- **Required Tech Stack:** PyTorch, TRL (Transformer Reinforcement Learning), Hugging Face Accelerate.
- **Task Description:** Build a preference optimization pipeline using a dataset of prompt-response pairs labeled as "chosen" and "rejected." Students must configure a dual-model memory landscape tracking both the active policy model and the frozen reference model to calculate relative log-likelihood losses during gradient updates.

Task 14: Custom CUDA Kernel Integration for Accelerated Activation Functions

- **Objective:** Bridge the gap between high-level Python scriptability and raw GPU hardware execution by writing and compiling custom, high-speed CUDA kernels.
- **Required Tech Stack:** C++, CUDA Toolkit, PyTorch C++ Extensions (cpp_extension).

- **Task Description:** Students must write a C++ header and CUDA kernel (`0.cu` file) to execute a vectorized SwiGLU activation function directly on the GPU thread grid. They must compile the extension using PyTorch's custom compiler bindings and benchmark its computational speedup against standard PyTorch activations.

Task 15: Enterprise LLM Gateway with Dynamic Rate-Limiting and Fallback Governance

- **Objective:** Design a robust, production-ready gateway to manage API quotas, model fallbacks, latency tracking, and request queuing under simulated peak loads.
- **Required Tech Stack:** FastAPI, Redis, Docker, Prometheus, Grafana.
- **Task Description:** Students will design a containerized microservice gateway acting as an intermediary between clients and multiple back-end LLM providers. The gateway must enforce token-bucket rate limiting via Redis, implement automated model failovers (e.g., routing to a secondary model if the primary model returns a `5xx` status code), and export structural performance metrics to Prometheus.